

INGENIERÍA DE SOFTWARE · ITLA

Práctica

Test-Driven Development (TDD)

Proyecto: Máquina de Café

n-Refactor

NUnit

C#

Refactoring

CI/CD

TDD consiste en construir primero los **casos de prueba** y, a partir de la técnica de refactorización, desarrollar el producto de software. El código solo se escribe para hacer pasar una prueba que ya falla.

1. ¿Qué es TDD? — Ciclo Red-Green-Refactor
2. Configurar el proyecto en Visual Studio
3. Casos de prueba — Máquina de Café
4. Implementación paso a paso
5. Buenas prácticas y checklist final

■ **Cierre:** martes 23 de junio de 2026, 11:59 PM

01 ¿Qué es TDD?

Test-Driven Development

Test-Driven Development (TDD) es una metodología de desarrollo donde el programador **escribe una prueba automatizada antes de escribir el código**. El proceso se repite en ciclos cortos hasta que toda la funcionalidad esté implementada.

El Ciclo TDD: Red → Green → Refactor



Descripción de cada fase

RED — Prueba fallida



Escribe una prueba que describa el comportamiento deseado. La prueba DEBE fallar porque el código aún no existe.

GREEN — Código mínimo



Escribe el código MÍNIMO para que la prueba pase. No importa si es feo — solo que funcione.

REFACTOR — Mejorar



Mejora el diseño: elimina duplicación, aplica buenas prácticas. Las pruebas DEBEN seguir pasando.

Beneficios del TDD

- **Alta cobertura de pruebas** desde el primer día — sin deuda técnica en testing.
- **Diseño más limpio** — TDD fuerza código modular con bajo acoplamiento.
- **Confianza al refactorizar** — si las pruebas pasan, el cambio no rompió nada.
- **Documentación viva** — los tests describen exactamente qué debe hacer el sistema.
- **Detección temprana de errores** — los bugs se encuentran en segundos, no en días.

02 Configurar el Proyecto

Solución con 2 proyectos en Visual Studio

Para TDD en C# necesitas una solución con **dos proyectos**: el proyecto principal con la lógica de negocio y un proyecto de pruebas con NUnit.

1 Crear la Solución

File → New → Project → Solution. Nombre: MaquinaCafeTDD

2 Agregar proyecto de producción

Add → New Project → Console App (.NET 8). Nombre: MaquinaCafe

3 Agregar proyecto de pruebas

Add → New Project → NUnit Test Project (.NET 8). Nombre: MaquinaCafe.Tests

4 Agregar referencia

En MaquinaCafe.Tests → Add Project Reference → seleccionar MaquinaCafe

5 Instalar NuGet

Instalar: NUnit, NUnit3TestAdapter, Microsoft.NET.Test.Sdk en el proyecto de pruebas

6 Ejecutar pruebas

Test → Run All Tests (Ctrl+R, A). Verificar que aparecen en el Test Explorer

Estructura de la solución

```
Estructura del proyecto
1 MaquinaCafeTDD/
2   MaquinaCafe/           # Proyecto principal
3     MaquinaCafe.cs       # Clase principal
4     Program.cs
5
6   MaquinaCafe.Tests/     # Proyecto de pruebas
7     MaquinaCafeTests.cs  # Todos los casos de prueba
8
9   MaquinaCafeTDD.sln
```

03 Casos de Prueba — Máquina de Café

Red phase: escribir primero

A continuación están los 8 casos de prueba ordenados por complejidad creciente. **Escríbelos todos primero** — deben fallar antes de implementar el código.

Regla de oro: NUNCA escribas código de producción antes de tener una prueba fallida que lo justifique.

TC-01 Insertar monedas

Escenario: La máquina acumula el saldo al insertar monedas.

Assert: `InsertarMoneda(25) → Saldo == 25`

TC-02 Seleccionar bebida con saldo suficiente

Escenario: Con saldo suficiente se puede seleccionar una bebida.

Assert: `InsertarMoneda(100) + SeleccionarBebida('Cafe') → true`

TC-03 Saldo insuficiente

Escenario: Con saldo insuficiente la selección es rechazada.

Assert: `InsertarMoneda(50) + SeleccionarBebida('Cafe') → false`

TC-04 Devolver cambio

Escenario: La máquina devuelve el cambio correcto tras dispensar.

Assert: `InsertarMoneda(150) + SeleccionarBebida('Cafe') → ObtenerCambio() == 50`

TC-05 Bebida no disponible

Escenario: Seleccionar una bebida inexistente lanza excepción.

Assert: `SeleccionarBebida('Jugo') → ArgumentException`

TC-06 Menú de bebidas

Escenario: El menú incluye Café=100, Té=75 y Agua=50.

Assert: `ObtenerMenu()` retorna 3 bebidas con sus precios

TC-07 Devolver monedas

Escenario: Devolver monedas reinicia el saldo a cero.

Assert: `InsertarMoneda(100) + DevolverMonedas() → Saldo == 0`

TC-08 Stock agotado

Escenario: No se puede dispensar una bebida sin stock.

Assert: `Stock=0 → SeleccionarBebida('Cafe') retorna false`

04 Implementación Paso a Paso

Ciclo RED → GREEN → REFACTOR

Seguimos el ciclo TDD para implementar la Máquina de Café. Primero escribimos los tests (RED), luego el código mínimo (GREEN) y finalmente mejoramos (REFACTOR).

Fase RED — Escribir los primeros tests

```
MaquinaCafeTests.cs — Fase RED
1 using NUnit.Framework;
2
3 [TestFixture]
4 public class MaquinaCafeTests
5 {
6     private MaquinaCafe _maquina;
7
8     [SetUp]
9     public void Inicializar() => _maquina = new MaquinaCafe();
10
11     // TC-01
12     [Test]
13     public void InsertarMoneda_DebeAcumularSaldo()
14     {
15         _maquina.InsertarMoneda(25);
16         Assert.AreEqual(25, _maquina.Saldo);
17     }
18
19     // TC-02
20     [Test]
21     public void SeleccionarBebida_SaldoSuficiente_RetornaTrue()
22     {
23         _maquina.InsertarMoneda(100);
24         Assert.IsTrue(_maquina.SeleccionarBebida("Cafe"));
25     }
26
27     // TC-03
28     [Test]
29     public void SeleccionarBebida_SaldoInsuficiente_RetornaFalse()
30     {
31         _maquina.InsertarMoneda(50);
32         Assert.IsFalse(_maquina.SeleccionarBebida("Cafe"));
33     }
34 }
```

Fase GREEN — Implementación mínima

```
MaquinaCafe.cs — Fase GREEN
1 public class MáquinaCafe
2 {
3     public int Saldo { get; private set; }
4
5     private Dictionary<string, int> _menu = new() {
6     private Dictionary<string, int> _menu = new() {
7         { "Cafe", 100 }, { "Te", 75 }, { "Agua", 50 }
8
9     public void InsertarMoneda(int monto) => Saldo += monto;
10
11    public bool SeleccionarBebida(string bebida)
12    {
13        if (!_menu.ContainsKey(bebida)) throw new ArgumentException("No existe");
14        if (Saldo < _menu[bebida]) return false;
15        Saldo -= _menu[bebida]; return true;
16    }
17
18    public int ObtenerCambio() { var c=Saldo; Saldo=0; return c; }
19    public void DevolverMonedas() => Saldo=0;
20    public Dictionary<string,int> ObtenerMenu() => _menu;
21 }
```

Fase REFACTOR — Mejorar el diseño

```
MaquinaCafe.cs — Fase REFACTOR
1 public class Bebida(string Nombre, int Precio, int Stock); // record
2
3 public class MaquinaCafe
4 {
5     private int _saldo;
6     private readonly Dictionary<string, Bebida> _menu;
7
8     public int Saldo => _saldo;
9
10    public MaquinaCafe()
11    {
12        _menu = new() {
13            ["Cafe"] = new("Cafe", 100, Stock: 10),
14            ["Te"] = new("Te", 75, Stock: 10),
15            ["Agua"] = new("Agua", 50, Stock: 10),
16        };
17    }
18
19    public void InsertarMoneda(int monto) => _saldo += monto;
20
21    public bool SeleccionarBebida(string nombre)
22    {
23        var b = ObtenerBebidaOLanzar(nombre);
24        if (_saldo < b.Precio || b.Stock == 0) return false;
25        _saldo -= b.Precio;
26        _menu[nombre] = b with { Stock = b.Stock - 1 };
27        return true;
28    }
29
30    private Bebida ObtenerBebidaOLanzar(string n)
31        => _menu.TryGetValue(n, out var b) ? b : throw new ArgumentException($"No existe {n}");
32
33    public int ObtenerCambio() { var c=_saldo; _saldo=0; return c; }
34    public void DevolverMonedas() => _saldo=0;
35    public Dictionary<string,Bebida> ObtenerMenu() => _menu;
36 }
```

Después del Refactor ejecuta TODAS las pruebas. Si alguna falla, el refactor introdujo un bug. Revierte y empieza de nuevo.

05 Buenas Prácticas y Checklist

Antes de entregar la práctica

Nombres para los tests — Patrón AAA

```
1 // Patrón de nombre: Metodo_Escenario_ResultadoEsperado
2
3 // Estructura interna: Arrange - Act - Assert
4 [Test]
5 public void SeleccionarBebida_SaldoExacto_DescuentaSaldo()
6 {
7     // ARRANGE
8     var maquina = new MaquinaCafe();
9     maquina.InsertarMoneda(100);
10
11     // ACT
12     var resultado = maquina.SeleccionarBebida("Cafe");
13
14     // ASSERT
15     Assert.IsTrue(resultado);
16     Assert.AreEqual(0, maquina.Saldo);
17 }
```

Checklist — Antes de entregar

- **Tests escritos primero (RED)**
¿Escribiste todos los 8 tests antes de implementar MaquinaCafe.cs? ¿Todos fallaban al principio?
- **Todos los tests en verde (GREEN)**
¿Pasan los 8 casos de prueba TC-01 al TC-08 en el Test Explorer?
- **Refactor aplicado**
¿El código está limpio, sin duplicaciones, bien nombrado y con responsabilidades claras?
- **Cobertura completa**
¿Cada método público de MaquinaCafe tiene al menos una prueba que lo cubra?
- **Commits separados**
¿Tienes commits diferenciados para la fase RED, GREEN y REFACTOR?
- **README actualizado**
¿El README explica qué es TDD, el ciclo aplicado y cómo ejecutar el proyecto y las pruebas?

Recuerda: TDD no es solo testing — es una disciplina de diseño. Cada prueba te fuerza a pensar en la interfaz pública de tu clase antes de implementarla, produciendo código más limpio, modular y mantenible.